Ders 3 Genişletilmiş Notu: Prototype, Singleton, Adapter ve Bridge Kalıpları

Ders 3 Genişletilmiş Notu: Prototype, Singleton, Adapter ve Bridge Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Prototype Kalıbı: Amaç ve Örnek Problem
- 2. Aynı Problemi Başka Yaratımsal Kalıplarla Çözmek
- 3. Prototype Kalıp Yapısı ve Uygulanabileceği Durumlar
- 4. Prototype Tartışmaları: Sığ ve Derin Kopyalama
- 5. Prototype Uygulama Örneği: Oyun İçin Level Editörü
- 6. Singleton Kalıbı
- 7. Yapısal Kalıplara Geçiş: Adapter Kalıbı
- 8. Adapter Uygulama Örneği: Hazır Bir Daire Sınıfını Uyarlamak
- 9. Bridge Kalıbı
- 10. Bridge Uygulama Örneği: İki Çizim Kitaplığıyla Şekiller
- 11. Çıkarılan Dersler
- Örnek Sorular
- Dersin Toparlaması

1. Prototype Kalıbı: Amaç ve Örnek Problem

1.1 Amaç

Önceki derste yaratımsal kalıplardan Abstract Factory, Factory Method ve Builder işlenmişti. Bu ders, yaratımsal grubun sıradaki kalıbı olan Prototype ile başladı. Slayttaki amaç şöyledir (ekrandan, 00:10:00): oluşturulacak nesnelerin türlerini belirlemek için bir prototip nesne kullanmak ve yeni nesneleri bu prototipi kopyalayarak oluşturmak.

Hoca, bu amaç doğrultusunda çalışan, başlı başına prototip temelli programlama dilleri de olduğunu söyledi. Prototype kalıbı, o dillerin çözmeye çalıştığı problemleri sınıf temelli dillerde çözen bir kalıp olarak düşünülebilir.

> **Ek bilgi (derste anlatılmadı):** Prototip temelli dillere büyük olasılıkla Self ve JavaScript örnek gösterilebilir. Bu dillerde sınıf kavramı yerine (ya da onun altında) nesneler doğrudan başka nesnelerden kopyalanarak ya da onlara bağlanarak türetilir.

1.2 Örnek problem: müzik besteleme programı

GoF kitabındaki örnekte bir müzik besteleme programı yazılacaktır. Bunun için grafik uygulamalarına yönelik, elimizde hazır duran bir çerçeve program (framework) kullanılacaktır. Slayttaki bilgiye göre (ekrandan, 00:10:00) çerçeve programında:

- soyut bir `Graphic` sınıfı,
- çeşitli kontrol düğmelerinin yer alabileceği paletleri simgeleyen soyut bir `Tool` sınıfı,

- `Tool`'dan kalıtımla türetilmiş ve yeni grafik şekillerini çizim alanına eklemeye yarayan bir `GraphicTool` sınıfı

vardır. Notaları çizim alanına eklemek için `GraphicTool` sınıfı kullanılabilir. Hoca, notaların sayısının belli olduğunu hatırlattı: yedi nota (do, re, mi, fa, sol, la, si) ve bunların içi dolu, içi boş, tekli, çiftli gibi birkaç varyasyonu vardır. Literatüre yeni bir nota şekli eklenmesi söz konusu olmadığına göre örnek uzayı bellidir.

Sorun şudur: çerçeve programının çeşitli nota sınıfları hakkında önceden bilgisi olamaz, çünkü çerçeve yazılırken bu sınıflar yoktu. O hâlde `GraphicTool` sınıfı, bilmediği sınıflardan çizim nesnelerini nasıl oluşturacaktır?

1.3 Çözüm

Slayttaki örnek çözüm şöyledir (ekrandan, 00:13:00):

```
Tool (soyut)                          Graphic (soyut)
  Manipulate()                          Draw(Position)
    ^                                   Clone()
    |-- RotateTool: Manipulate()         ^
    |-- GraphicTool: Manipulate() ---->|  (prototype üyesi, GraphicTool -> Graphic)
                                        |-- Staff: Draw(Position), Clone()
                                        |-- MusicalNote (soyut)
                                              |-- WholeNote: Draw(Position), Clone()
                                              |-- HalfNote:  Draw(Position), Clone()

GraphicTool.Manipulate() notu:
  p = prototype->Clone()
  while (user drags mouse) {
      p->Draw(new position)
  }
  insert p into drawing

WholeNote.Clone() ve HalfNote.Clone() notu: return copy of self
```

`Tool` sınıfının soyut `Manipulate()` metodu vardır. Örneğin `RotateTool`, kontrol çubuğundan seçilen bir şekli döndürmek için kullanılan bir araçtır. `GraphicTool` ise sürükle-bırak biçiminde şekil taşıma ve ekleme amaçlı bir araçtır. Onun somut `Manipulate()` metodu şöyle çalışabilir: öncelikle bir `prototype` üyesi vardır. Yeni bir eleman eklenmek istendiğinde prototip nesnenin kopyası oluşturulur, kullanıcı fareyi sürükledikçe kopya yeni konumda çizdirilir, bırakıldığında da kopya çizime eklenir.

Hoca, şemadaki ilişkinin aslında bire çok sahiplik (parça-bütün) ilişkisi gibi düşünülebileceğini söyledi: işin esprisi, her yeni eleman için prototipin yeni bir kopyasını oluşturmaktır. Bir notanın anlamını şekli (dolu, boş, ters, düz) ve porte (staff) üzerindeki konumu belirler; porte, sol anahtarı ve beş çizgiden oluşur, notanın çizgilerin üstünde ya da arasında olması anlam taşır.

Hoca şema üzerinde bir kırmızı çizgi çizerek ayrımı gösterdi: çizginin üstü elimizdeki hazır çerçeveye (`Tool`, `Graphic`), altı ise bizim yazacağımız müzik uygulamasına aittir. `Graphic` sınıfında bir grafik nesnesini belli bir konuma çizmek için soyut `Draw(Position)` ve bir kopyasını oluşturmak için soyut `Clone()` tanımlıdır. Temel grafik bileşenleri porte (`Staff`) ve soyut `MusicalNote`'tur; tam nota (`WholeNote`), yarım nota (`HalfNote`) gibi somut alt sınıflar, yukarıdaki soyut `Clone()` metodunu kendi kopyalarını döndürecek biçimde somutlaştırır. Kullanıcı arayüzü, araç çubuğundan bir nesne seçilip çizim alanına bırakılacak biçimde kurulabilir.

2. Aynı Problemi Başka Yaratımsal Kalıplarla Çözmek

Hoca, bu problemin başka kalıplarla da çözülebileceğini, ama zorlukları olduğunu tartıştı.

Abstract Factory ile: Abstract Factory'de ürün aileleri ve her ailenin bireysel üyeleri vardı. Burada aileler nota türlerine göre mi (tam nota, yarım nota), yoksa notalara göre mi (do, re, mi, fa) kurulmalıdır? Hoca müzik alanında deneyimi olmadığı için kesin konuşamayacağını

söyledi, ama bir ölçüt önerdi: hangisinin sayısı azsa onu aile yapmak. Tam, yarım, ters, düz biçiminde dört nota türü, buna karşılık yedi nota vardır; bu durumda "dört ailenin yedi üyesi" demek daha mantıklıdır. Örneğin bir tam nota fabrikası ve bir yarım nota fabrikası olur; tam nota fabrikasından "do" istenince tam nota do verilir.

Factory Method ile: benzer biçimde kurulabilir. Her iki çözümün de sorunu, her nota ya da her nota türü için ayrı bir fabrika sınıfı gerektirmesidir: yedi nota için yedi fabrika, dört nota türü için dört fabrika. Bir uygulama için bu kadar çok fabrika yazmak hocaya az kullanışlı geldi. Hoca, öğrencilerin aklına başka çözümler gelirse bunları paylaşabileceklerini söyledi.

3. Prototype Kalıp Yapısı ve Uygulanabileceği Durumlar

3.1 Kalıp yapısı

Slayttaki kalıp yapısı şöyledir (ekrandan, 00:17:30):

```
Client                      prototype       Prototype (soyut)
  Operation()  ---------------------------->  Clone()  (soyut)
    not: p = prototype->Clone()                 ^
                                                |-- ConcretePrototype1: Clone()  -> return cop
                                                |-- ConcretePrototype2: Clone()  -> return cop
```

Kopyalanıp çoğaltılacak soyut bir `Prototype` sınıfı ve onun soyut `Clone()` metodu vardır. Klonlamanın derin ya da sığ kopya gibi birçok ayrıntısı olduğu için hangisi işimize geliyorsa o kullanılır. Prototipi kullanacak `Client`, bir işlem (`Operation()`) yapmadan önce sahip olduğu prototip nesnenin kopyasını oluşturur, sonra işleri o kopya üzerinden yürütür.

İşlerin farklı biçimlerde yapılması gerekiyorsa birden fazla somut prototip sınıfı olabilir. Ama illa birden fazla olması gerekmez; tek bir somut prototip de yeterli olabilir. Kardeşler ancak bazı metotlarda birbirinden farklılaşacaksa eklenir.

3.2 Uygulanabileceği durumlar

Slayttaki maddeler şöyledir (ekrandan, 00:20:00):

- Bir sistemin, kendisine ait ürünlerin nasıl oluşturulduğu, birleştirildiği ve sunulduğundan bağımsız olmasının istendiği durumlar,
- ve şu durumlardan biri söz konusu ise (Factory Method kalıbından farklılaşmayı sağlıyor):
  - Oluşturulacak sınıflar çalışma anında belirlenecekse,
  - Ürünlerin sınıf hiyerarşisi ile paralel bir factory hiyerarşisi kurmak istenmiyorsa,
  - Ürün sınıfının örneklerinin durum uzayı sınırlı ise.

İlk madde Abstract Factory ve Factory Method'da da geçen ortak koşuldur. Prototype'ı Factory Method'dan ayıran, alt maddelerdeki koşullardır. Hoca bunları tek tek açıkladı:

- **Sınıfların çalışma anında belirlenmesi:** Abstract Factory ya da Factory Method'da ürünler ve ürün aileleri bellidir. Gerçi müzik örneğinde de nispeten bellidir. Ama birazdan görülecek oyun örneğinde bölüm sonu canavarı tasarımı gibi konularda farklı canavar türleri devreye girebilir; hatta oyunun zorluğu oyun sırasında değiştirilebilir. Burada "sınıf" kelimesi programlama dilindeki class anlamında değil, problem alanındaki mantıksal bileşen türleri anlamındadır: bölüm sonu canavarı, standart düşman, uçan düşman, uzaktan saldıran düşman, büyücü düşman gibi.
- **Paralel factory hiyerarşisi:** Factory Method kullanılırsa ürünlerin sınıf hiyerarşisine paralel bir factory hiyerarşisi kurmak gerekir. Hiyerarşi karmaşıksa bu istenmeyebilir.
- **Sınırlı durum uzayı:** Prototiplemede nesnenin kendisi kopyalanır. Çok fazla durum varsa bu durumları prototipler üzerinden belirtmek kullanışsız olabilir. Durum uzayı sınırlıysa oluşturulacak prototip sayısı da az olur ve iş kolaylaşır. Hoca, nota örneğinde olduğu gibi durum uzayı sınırlıysa belki Factory Method'un da kullanışlı olabileceğini, bunun üzerine biraz düşünmesi gerektiğini söyledi.

Hoca, üç alt maddenin hepsinin aynı anda geçerli olmasının gerekmediğini, birinin söz konusu olmasının Prototype'ı tercih etmek için yeterli avantajı sunduğunu vurguladı.

3.3 Dil yeteneklerinin etkisi

Slayttaki ikinci madde (ekrandan, 00:20:00): bu kalıp, C++ gibi sınıfların doğrudan işlenebilecek varlıklar olmadığı dillerde daha çok yarar sağlar. Smalltalk gibi geç ilişkilendirme (late binding) ve Java gibi yansıtma (reflection) yetenekleri olan dillerde bu kalıp yerine dilin yetenekleri kullanılabilir.

Hoca, derste çok dile taşınabilir örneklerle çalışıldığını, ama kullanılan dilin yeteneklerinin bazı işleri kolaylaştırıp zorlaştırabildiğini söyledi. Bildiği kadarıyla C++'ta böyle kütüphaneler yoktur. Java'da ise geniş bir yansıtma (öz inceleme) yeteneği vardır: herhangi bir sınıfı alıp hangi üyeleri, hangi metotları olduğunu, hangi metodun kaç parametre alıp ne döndürdüğünü öğrenebilir, bir metoda işaretçi alıp başka bir yerden çalıştırabilirsiniz. Hatta dilin kendisinde olmasa bile bazı çatılarla bir sınıfın byte kodunu çalışma anında değiştirebilirsiniz. Bu, Prototype kalıbına gerek kalmadan çok daha fazlasını yapmayı sağlar, ama dile sıkı bağlı bir çözümdür.

Hoca, Smalltalk'un nesneye yönelimde ne gibi farklı bakış açıları kazandırdığını merak edenlerin incelemesini önerdi. Smalltalk'ta geç ilişkilendirme vardır: bir değişkeni türünü belirtmeden tanımlar gibi nesne tanımlanır ve türü daha sonra belirlenebilir. Piyasada Smalltalk ile yazılmış pek program yoktur, ama incelemek insana bakış açısı kazandırır.

4. Prototype Tartışmaları: Sığ ve Derin Kopyalama

Slayttaki tartışma maddeleri şöyledir (ekrandan, 00:27:50):

- Sığ kopyalama veya derin kopyalama (shallow or deep copy)
  - Sığ kopyalama yetersiz kalabilir
  - Çevrimsellik derin kopyalamada sorun çıkartır.
- Klonların üyelerine değer atanması için Prototype sınıfında ek metotlar tanımlamak gerekebilir.

Klonlamada farklı farklı şeyler devreye girebildiği için `Clone()`'un soyut tanımlanıp ayrıntılarının somut sınıflarda belirlenmesi yararlıdır. İlk dikkat edilecek şey, kopyalamanın derin mi sığ mı yapılacağıdır.

Hoca bunu bir çalışan nesnesiyle açıkladı. Çalışanın şirketi, amiri, iş telefonu gibi sahiplik ilişkisi içinde olduğu farklı nesneler olsun. Çalışan nesnesini kopyalarken onun bütün üye alanlarındaki nesneleri de kopyalıyorsanız derin kopyalama, yalnız çalışan nesnesinin kendisini kopyalıyorsanız sığ kopyalama yapmış olursunuz.

İlk bakışta derin kopyalama her zaman daha iyi gibi görünebilir. Ama bütünleşik bir iş uygulamasında çalışanlar için ayrı, amirler için ayrı kayıtlar tutulur; amir de bir çalışan olduğu için veri tabanında ayrıca kaydı vardır. Çalışanı kaydederken amirini de derin kopyalamak, zaten ayrıca kaydedilecek bir nesne için programa boşuna iş yaptırmak ve onu yavaşlatmaktır. Bu durumda nesneleri sığ kopyalayıp ilişkileri sonra düzenlemek doğru bir düşüncedir. Hangi kopyalama mantığı işe yarıyorsa o seçilmelidir.

Derin kopyalamada çevrimselliğe de dikkat edilmelidir: çalışan → çalışanın amiri → amirin altındakiler → yeniden çalışanın kendisi biçiminde bir döngü varsa derin kopyalama sonsuz döngüye girebilir.

Klonlama sırasında bir kopya kurucu (copy constructor) yazmak ya da nesnenin durum bilgisinin tamamını alıp onu kopyalayan metotlar bulundurmak yararlı olabilir. Ayrıca klonların değişecek üyelerine değer atanması için soyut `Prototype` sınıfına ek metotlar kodlamak gerekebilir, çünkü bütün prototip nesneleri eş biçimli (aynı arayüz üzerinden) ele alınacaktır.

5. Prototype Uygulama Örneği: Oyun İçin Level Editörü

5.1 Problem

Bu örnek Head First Design Patterns kitabından alınmıştır. Slayttaki problem şöyledir (ekrandan, 00:30:00):

- Bir bilgisayar oyunu için level editörü hazırlanıyor.

- Haritaya çok çeşitli türlerden canavarları kolayca ekleyebilmek istiyoruz.
- Ama bir canavar oluşturmak kolay iş değil, bir sürü durum bilgisi var: HP, DEX, AGL, LUCK, CHR vb.

Canavarlar sürükle-bırak gibi yollarla kolayca eklenmek, yeni canavar türleri de kolayca oluşturulmak isteniyor. Ama bir canavarın en az beş ayrı durum bilgisi vardır. Hoca bunları açıkladı: can puanı (HP), saniyede ne kadar çok vuruş yapabileceği gibi beceriyi tutan DEX, çeviklik (agility; vurulması zor olabilir ya da hızlı kaçabilir), şans (luck; şanslı canavarlar daha yüksek, kritik hasar verebilir) ve karizma (CHR; örneğin gücü sizden düşük bir canavarı savaşmak yerine ekibinize katılmaya ikna edebilirsiniz). Bu değerlerin değişik kombinasyonları değişik canavar ya da oyuncu karakter türleri ortaya çıkarır; türün adı da bir `type` alanında tutulur.

Akla gelen çözüm şudur: bir canavar türünü prototip olarak tutmak, sonra onu kopyalayıp özelleştirerek farklı canavarları kolayca oluşturmak.

5.2 Tasarım

Slayttaki sınıf şeması şöyledir (ekrandan, 00:30:00):

```
<<Abstract>> Monster                          [Prototype]
  # HP, DEX, AGL, LUCK, CHR : int
  # type : String
  + clone() : Monster {abstract}
  + getType() : String
  # Monster(int, int, int, int, int)
      ^
      |
GenericMonster                                [ConcretePrototype]
  + clone() : Monster
  + GenericMonster(int, int, int, int, int)

MonsterRegistry                               1 ----> * Monster
  + findMonster( type : String ) : Monster
  + registerMonster( Monster )

MainApp                                       [Client]
  - - -> MonsterRegistry, - - -> GenericMonster  (kullanma)
```

`Monster` sınıfında `clone()` metodu bulunmak zorundadır ve soyuttur. Statlar için getter metotları ve beş parametreyi alan bir kurucu vardır. Dikkat edilecek nokta kurucunun `protected` olmasıdır. Kalıbın arkasındaki fikir kopya üzerinden çalışmaktır; kurucu çağrılarak sıfırdan nesne oluşturulmak istenmez. Alt sınıfların kurucuyu çağırması gerekebileceği için `private` kurucu işe yaramaz, ama `protected` çok işe yarar: alt sınıflara public, diğer sınıflara private gibi davranır.

Hoca, kalıbın kendi tanımında kurucu hakkında bir şey yazmadığını, ama bunu düşünmenin yararlı olduğunu söyledi: klonlanacak varlığın sınıfına public değil protected bir kurucu eklemek yararlıdır.

`GenericMonster`, `Monster`'dan kalıtımla türetilmiş somut prototip sınıfıdır ve `clone()` metodunu kodlar. Kaynakta bu sınıfın kurucusu public yapılmıştır. Hoca, istemcilere zaten soyut `Monster` işaretçisi verileceğini, somut alt sınıf işaretçisi verilmeyeceğini hatırlattı: kalıpta `Client` yalnız `Prototype` ile muhataptır, somut prototiplerle bir ilişkisi yoktur. Ama istemci aynı paketten `Monster`'a ulaşabiliyorsa `GenericMonster`'a da ulaşabilir; bu yüzden belki onun kurucusunu da farklı tanımlamakta yarar olduğunu not düştü.

Oluşturulan farklı canavar türleri daha sonra kaydedilmek ve onlara yeniden erişilmek istendiği için bir `MonsterRegistry` sınıfı oluşturulur. Burada canavar nesneleri bir map yapısında tutulur: anahtar alan canavarın türüdür. `findMonster` metodu anahtarı alıp ilgili canavar nesnesini döndürür; `registerMonster` yeni canavarları kaydeder. `MainApp` kalıptaki `Client` rolündedir; oyun editörü odur.

5.3 Kod

`Monster` sınıfı (ekrandan, 00:35:10):

```
package dp.prototype.example;
public abstract class Monster {
    protected int HP, DEX, AGL, LUCK, CHR;
    protected String type;

    public abstract Monster clone( );

    protected Monster(int hp, int dex, int agl, int luck, int chr) {
        HP = hp; DEX = dex; AGL = agl; LUCK = luck; CHR = chr;
    }
    public String getType() { return type; }
    public int getHP() { return HP; }
    protected void setHP(int hP) { HP = hP; }
    public int getDEX() { return DEX; }
    protected void setDEX(int dEX) { DEX = dEX; }
    public int getAGL() { return AGL; }
    protected void setAGL(int aGL) { AGL = aGL; }
    public int getLUCK() { return LUCK; }
    protected void setLUCK(int lUCK) { LUCK = lUCK; }
    public int getCHR() { return CHR; }
    protected void setCHR(int cHR) { CHR = cHR; }
    protected void setType(String type) { this.type = type; }
}
```

Slayttaki not: “protected setter’lar ile erişimin tek yolu Register olur, tercih ederim.” Hoca, setter’ları protected yapmanın mantıklı olduğunu söyledi. Amaç, registry’den bir canavar alınıp kategorisi dışında statlarının değiştirilmesini engellemektir. Bunu açıklarken protected’ın alt sınıflara ve paketteki diğer sınıflara public olarak göründüğünü hatırlattı.

> **Ders sonrası not:** Hoca kurucuyu anlatırken (00:31:32) protected için “alt sınıflara public, diğer sınıflara private gibi davranır” dedi, setter’ları anlatırken (00:35:44) aynı paketteki sınıfları da ekledi. Java’da doğrusu ikinci anlatımdır: `protected` üyeye alt sınıflar ve aynı paketteki bütün sınıflar erişebilir. C++’ta paket kavramı olmadığı için ilk anlatım C++’a uyar.

Hoca, kodda üye alanların protected tanımlandığına dikkat çekti. Bu sayede alt sınıflar onlara doğrudan erişebiliyor, ama bu hocanın normalde tercih ettiği bir yol değildir; üst sınıfın üye alanlarını private tutmayı tercih eder. “Sen dediğini yap, yaptığını yapma” değil, “dediğini de yap, yaptığını da yap” olsun diyerek bunu slaytta düzelteceğini not düştü.

`GenericMonster` ve `MonsterRegistry` (ekrandan, 00:38:30):

```
package dp.prototype.example;
public class GenericMonster extends Monster {
    public GenericMonster(int hp, int dex, int agl, int luck, int chr) {
        super(hp, dex, agl, luck, chr);
        type = "Generic Monster";
    }
    public Monster clone( ) {
        return new GenericMonster(HP, DEX, AGL, LUCK, CHR);
    }
}
public class MonsterRegistry {
    private HashMap<String, Monster> monsters;
    public MonsterRegistry() { monsters = new HashMap<String,Monster>(); }

    public Monster findMonster( String type ) {
        return monsters.get( type );
    }
```

```
    public void registerMonster( Monster m ) {
        monsters.put( m.getType(), m );
    }
}
```

`GenericMonster` kurucusu parametreleri alıp üst sınıfın kurucusunu çağırır ve türünü "Generic Monster" olarak belirler. `clone()` aynı stat değerlerine sahip yeni bir `GenericMonster` nesnesi oluşturur. `MonsterRegistry`, Java'daki generic `HashMap` yapısını kullanır: anahtar türü `String`, değer türü `Monster`'dır. `get` anahtarı verip nesneyi döndürür, `put` yeni bir nesneyi anahtarıyla ekler.

`MainApp` (ekrandan, 00:40:00):

```
public class MainApp {
    public static void main(String[] args) {
        MonsterRegistry reg = new MonsterRegistry();
        reg.registerMonster( new GenericMonster(10, 10, 10, 10, 10) );
        Monster mySpecialMonster =
                reg.findMonster("Generic Monster").clone();
        mySpecialMonster.setHP(100);
        mySpecialMonster.setType("Bolum sonu canavarı");
        reg.registerMonster(mySpecialMonster);
    }
}
```

Adım adım: önce bir registry oluşturulur. Ona bütün değerleri 10 olan genel bir canavar kaydedilir. Özel bir canavar oluşturmak için registry'den "Generic Monster" alınır ve `clone()` ile kopyası çıkarılır. Kopyanın canı 100'e yükseltilir, türü "Bolum sonu canavarı" yapılır ve registry'ye kaydedilir. Bundan sonra bu anahtarla istendiğinde registry o canavarı verir. Hoca, slayttaki metinde "Bolum" ile "canavarı" arasında Türkçe karakter tutarsızlığı olduğunu da fark etti.

5.4 Tasarım üzerine tartışma

Hoca, `main` içinde `GenericMonster` kurucusuna doğrudan erişildiğine dikkat çekti. Bu bir kolaylık sağlar, ama başlangıçta mutlaka bir canavar oluşturup registry'ye koymak gerekir. Önerdiği alternatif (ekrandan, 00:41:20): üst sınıfın kurucusu protected kalır; somut prototip sınıfının kurucusu da private yapılır. Registry onunla aynı paketteyse, ilk oluşturulduğunda map yapısını kurduktan sonra varsayılan bir canavar türünü kendisi oluşturup kaydedebilir. `main`'deki genel canavar oluşturma kısmı böylece registry'nin içine taşınır; yeni canavarlar o varsayılan canavarın kopyalanıp değiştirilmesi ve registry'ye geri yazılmasıyla elde edilir.

Soru (hocanın sorduğu itiraz): Nasıl olsa istediğimiz değerlere sahip yeni canavarlar oluşturabiliyoruz; o zaman neden doğrudan kurucuyla canavar oluşturmayalım? **Cevap:** Haklı bir itiraz. Canavar nesnelerine erişimin kısıtlanmasının yararlı olduğu gereksinimler eklemek istiyorsanız (kurucuları gizlemek, yalnız birkaç üyeyi değiştirerek registry'ye yeniden yazmaya izin vermek gibi), bu tasarım ona yönelik bir altyapı oluşturur. Çözülecek problemin özel gereksinimleri neyi gerektiriyorsa o yönde ilerlenir.

> **Hocanın vurgusu:** Bir tasarım kalıbını kullanmak onu motamot kullanmak demek değildir. Gereksinimlerinize göre kalıpları değiştirip özelleştirerek tasarımlarınızı kuracaksınız.

6. Singleton Kalıbı

6.1 Amaç ve örnekler

Slayttaki bilgiler şöyledir (ekrandan, 00:46:10):

- Amaç: bir sınıfın sadece bir örneğinin bulunmasını sağlamak ve bu nesneye ortak bir erişim noktası vermek.

- Örnek: bazı nesnelerin türünün tek örneği olmasının gerekli olduğu alanlar bulunabilir. Bir bilgisayarda birden fazla yazıcı tanımlı olabilmesine rağmen bir tek yazdırma biriktiricisi bulunur. Bir işletim sisteminde tek bir dosya sistemi ve pencere yöneticisi bulunur.

Kalıbın adı da buradan gelir: single, yani tek. Hoca, buna tahmin edilenden daha fazla yerde ihtiyaç duyulabileceğini söyledi. Yazdırma biriktiricisi (spooler) örneğinde uygulamalar yazdırılacak dosyaları biriktiriciye gönderir; biriktirici isteği hangi yazıcıya gidecekse ona yönlendirir. Yazıcıların tampon belleği sınırlıdır, işleri hemen basamazlar ve bekletmeleri gerekir; biriktirici bunun için tampon bellek ayırır. Bu işleri birden fazla süreç yaparsa tutarsızlıklar ve çelişkiler doğar.

6.2 İlk akla gelen çözüm ve sorunu

Slayttaki Çözüm 1 (ekrandan, 00:46:10): tek olacak nesne static olarak tanımlanır. Ama bu nesne farklı sınıflar tarafından kullanılacaksa, bunlardan hangisinin üyesi olacaktır?

Hoca bu soruyu açtı: nesne en çok hangi sınıf tarafından kullanılıyorsa ona konabilir. Ama ona erişmesi gereken farklı türden çok sayıda nesne varsa, bu kez ileride görülecek Feature Envy kod kusuru ortaya çıkar. Birden fazla türden nesnenin bu ortak nesneye ihtiyacı olduğu için bu yolla sorun bir türlü çözülemez.

6.3 Önerilen çözüm

Akla gelen sonraki fikir şudur: ortak paylaşılacak nesne apayrı bir sınıftan olsun ve o sınıf kendi örneğini oluşturmaktan ve paylaşmaktan sorumlu olsun. Bu, zaten Singleton kalıbının çözümüdür. Slayttaki önerilen çözüm (ekrandan, 00:50:00):

- Bir sınıfa, kendisinin tek bir örneğinin olmasını sağlama sorumluluğunu atamak:
  - Sınıfın kendi türünden static bir üyesi olur,
  - Sınıfın kurucusu private tanımlanarak kurucuya erişim engellenir,
  - Sınıfın bu üye için bir factory metodu bulunur.
- Kalıba adını veren bu sınıfın adı Singleton olarak belirlenmiştir.

```
Singleton
  - uniqueInstance : Singleton {static}
  - state : SingletonData
  - Singleton( ... )
  + getInstance() : Singleton {static}
  + singletonOperation()
  + getterMethods()

getInstance() notu:
  if( uniqueInstance == null ) {
      uniqueInstance = new Singleton( ... )
  }
  return uniqueInstance;
```

Sınıfın kendi türünden static üyesi kalıpta `uniqueInstance` olarak adlandırılmıştır. Kurucu private olduğu için dışarıdan birden fazla nesne oluşturulamaz. `uniqueInstance`'ı döndüren factory metodunun adı `getInstance()`'tır; `uniqueInstance` static olduğu için bu metodun da static tanımlanması gerekir. `getInstance()` içinde, nesne henüz ilklendirilmemişse ilklendirilir, sonra döndürülür.

6.4 Ek ayrıntılar

Hoca kalıbın karmaşık olmadığını, bu yüzden kod örneği yapmayacağını söyledi ve birkaç ayrıntıya değindi:

- İleride görülecek Strategy ve State kalıplarındaki somut alt sınıflar aynı zamanda birer Singleton adayıdır. Ama öyle olması şart değildir; derste o kalıpların örneklerinde böyle yapılmayacak, isteyen yapabilir.

- Singleton sınıflarının kalıtımla alt sınıfları oluşturulabilir; böylece tek nesne daha özelleşmiş biçimlerde de kullanılabilir. Ortak nesneyi bir static üye olarak ihtiyaç duyan sınıflardan birine koyan ilk çözümde bu esneklik yoktur.
- Singleton nesnesini yalnız tek bir türden nesne kullanacaksa ilk akla gelen çözümü kullanmakta sakınca yoktur.

Özetle kalıp, bir nesnenin kendi sınıfının tek örneği olmasının gerektiği ve bu nesnenin farklı sınıflardan nesnelerce kullanılması gerektiği durumlarda rahatlıkla kullanılabilir. Singleton ile yaratımsal kalıpların sonuna gelinmiştir.

> **Ek bilgi (derste anlatılmadı):** Notta görülen `getInstance()` biçimi "tembel ilklendirme" (lazy initialization) olarak bilinir. Çok iş parçacıklı programlarda iki iş parçacığı aynı anda `uniqueInstance == null` görürse iki nesne oluşabilir; bu yüzden Java'da büyük olasılıkla `synchronized`, statik ilklendirme ya da `enum` tabanlı Singleton gibi yollar önerilir.

7. Yapısal Kalıplara Geçiş: Adapter Kalıbı

7.1 Amaç ve benzetme

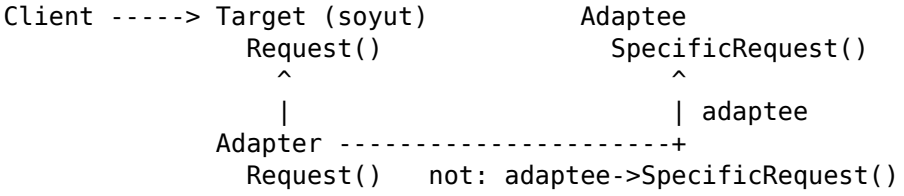
Aradan sonra yapısal (structural) kalıplara geçildi. İlk kalıp Adapter'dır. Slayttaki bilgiler (ekrandan, 01:10:00):

- Amaç: işlev açısından uyumlu ancak metot adları açısından uyumsuz bir istekçi sınıf ile bir sunucu sınıfı, birbirleri ile tek yönlü konuşturmak.
- Örnek: İngiliz standardındaki elektrik prizine Avrupa standardındaki cihazı takabilmek için bir adaptör kullanmak gerekir. Adapter kalıbı da aynı şekilde çalışır.

Kalıbın adı, kelimenin sözlük anlamlarından biriyle kullanılmıştır. Hoca, İngiltere'de (ve Kıbrıs'ta) farklı bir priz standardı olduğunu, bazı ülkelerde priz bacaklarının aralığının farklı olduğunu anlattı. Bilgisayarınızın şarj adaptörünü takmak istediğinizde o prizi doğrudan kullanamazsınız; araya koyduğunuz bir uyarlayıcı parça, Türk ya da Avrupa standardındaki fişinizi İngiliz standardındaki prize uygun hale getirir.

7.2 Kalıp yapısı

Slayttaki kalıp yapısı ve katılımcılar (ekrandan, 01:12:30):



- Adaptee: uyarlanması istenen sınıf (sunucu).
- Client: uyarlanacak sınıfa iş yaptırmak isteyen istekçi sınıf.
- Target: istekçinin konuşmak istediği arayüz.
- Adapter: uyarlama işlemini yapan, programcının yazacağı sınıf. Diğer sınıflar halihazırda mevcuttur.

Sunucu sınıf (`Adaptee`), işi yapmak için `SpecificRequest` metoduna sahiptir. İstemci ise kodunda bu iş için `Request` metodunu çağıracak biçimde yazılmıştır ve `Target` ile konuşmaktadır. Yapılacak işte bir sorun yoktur, yalnız metot imzası istenen gibi değildir. Çözüm: `Request` metodunu içeren `Target` sınıftan kalıtımla bir `Adapter` sınıfı oluşturulur ve ona istemcinin istediği `Request` metodu eklenir. Adaptörün, asıl işi yürütecek nesneye bir sahiplik ilişkisi vardır; `Request` metodunda yapılan tek şey, çağrıyı o nesneye `SpecificRequest` olarak iletmektir.

Hoca, bunun kalıbın en yalın hali olduğunu, `Target`'ın ya da `Request` metodunun illa soyut olmasının gerekmediğini söyledi. Kalıp tek odaklı ve karmaşık olmadığı için ayrı bir kod örneği yapmadı.

7.3 Uygulanabileceği an

Elimizdeki bir sınıfın bir istekçi tarafından kullanılmasını önleyen tek engel, istekçinin yolladığı mesaj ile sınıfın anladığı mesajın adlarının farklı olmasıdır. Hoca, parametrelerin sırasının farklı olması durumunun da buna eklenebileceğini, adaptörde beklenen sırayı çevirerek işin yaptırılabileceğini söyledi.

8. Adapter Uygulama Örneği: Hazır Bir Daire Sınıfını Uyarlamak

8.1 Problem

Slayttaki problem (ekrandan, 01:14:50):

- Bir çizim programı üzerinde çalışılıyor.
- Programın bir kısmı halihazırda gerçeklenmiştir (aşağıda solda).
- Daire kodunu yazmadan önce fark ettik ki, elimizde hazır bir daire sınıfı var.

```
Shape (soyut)                          ???          XXCircle
  +setLocation()                   <-------->        +setLocation()
  +getLocation()                                     +getLocation()
  +display()     (soyut)                             +displayIt()
  +fill()        (soyut)                             +fillIt()
  +setColor()    (soyut)                             +setItsColor()
  +undisplay()                                       +undisplayIt()
    ^
    |-- Point:  +display(), +fill(), +undisplay()
    |-- Line:   +display(), +fill(), +undisplay()
    |-- Square: +display(), +fill(), +undisplay()
```

Soyut `Shape` sınıfında konum için x-y koordinatının setter ve getter metotları vardır. Şekli çizmek için `display()` soyut olmak zorundadır, çünkü farklı şekiller çok farklı çizilir. Şeklin içini doldurmak (`fill()`) ve rengini değiştirmek de şekle göre farklı kodlanacağı için soyut metotlardır. Şekli ekrandan ya da grafik dosyasından silmek (`undisplay()`) da benzer biçimde şekle bağlıdır. Nokta, çizgi ve kare sınıfları yazılmıştır; sıra daire sınıfına gelmiştir.

Hoca, daire çizmenin kolay olmadığını hatırlattı. Ekran satır ve sütunlardan oluşan bir ızgaradır; tam yuvarlak çizilemez, daireye yaklaşan bir çokgen gibi bir şey çizilir. Yumuşatma (anti-aliasing) tekniğiyle bazı ara kareler rengin daha silik bir tonuyla doldurularak mükemmele daha yakın bir daire elde edilebilir.

Bunu kodlamaya geçerken önceki bir projede bu sınıfın zaten yazıldığını hatırlarsınız. Kaynak kodu da kaybolmuş olabilir; elinizde yalnız derlenmiş dosyalar (Java için class dosyası) vardır. Bu `XXCircle` sınıfını yeni proje için sıfırdan yazmak istemezsiniz. Ama onu doğrudan yerleştiremezsiniz: tasarımda her şeklin `Shape`'ten türetilmesi gerekir ve `XXCircle`'ın kaynak kodu değiştirilemediği için bu yapılamaz. Metot adları da farklıdır.

8.2 Çözüm

Slayttaki çözüm (ekrandan, 01:20:00):

```
class Circle extends Shape {
  ...
  private XXCircle myXXCircle;
  ...
  public Circle () {
    myXXCircle= new XXCircle();
  }

  void public display() {
     myXXCircle.displayIt();
  }
  ...
}
```

> **Ders sonrası not:** Slayttaki `void public display()` yazımı Java'da derlenmez; erişim belirleyicisi dönüş türünden önce gelir: `public void display()`.

Yeni projeye `Shape`'ten türetilen bir `Circle` sınıfı eklenir. Soyut metotları somutlaştırmak için `Circle`'a `XXCircle` türünden bir üye alan eklenir; kurucuda bu üye oluşturulur. Her metot kodlanırken, ikili dosyadaki sınıfın zaten kodlanmış olan karşılık gelen metodu çağrılır. Böylece `display` metodunun adının `displayIt` olması sorun çıkarmaz; `Circle`, mevcut kodu yeni çerçeveye uyduran (adapte eden) sınıf olur.

Hoca, şemada `Circle` ile `XXCircle` arasında dolu elmas (parça-bütün) gösterildiğini, ama burada sahiplik (içi boş elmas) ilişkisinin daha mantıklı olacağını söyledi; şemayı kaynaktan hazır aldığı için düzeltmediğini belirtti.

9. Bridge Kalıbı

9.1 Amaç

Bridge kalıbında amaç, nesnenin arayüzü ile nesnenin gerçeklemesini birbirinden ayırmaktır. Hoca bu cümlenin ilk bakışta çok iddialı durduğunu kabul etti: arayüz, nesnenin sahip olduğu metotlar, yani dışarıya verdiği hizmetlerdir; gerçekleme bunların kodlanmasıdır. Normalde bunlar ayrılmaz. Burada kastedilen gerçek bir ayırma değil, istemcileri gerçeklemeden soyutlamaktır.

Kalıbın alternatif adı Handle'dır. Hoca, Windows'un eski sürümlerinde Windows'a özgü kod yazarken çeşitli olayların handle'lar üzerinden ele alındığını, Windows'un handle mantığının bu kalıbın bir gerçeklemesi olduğunu söyledi.

9.2 Sorun

GoF örneğinde bir pencere çerçeve programının farklı masaüstü ortamlarında (ör. GNOME, KDE) çalışabilmesi istenmektedir. Slayttaki sorun şeması (ekrandan, 01:24:00):

```
Önce:                          Sonra:
Window                         Window
  |-- XWindow                    |-- XWindow
  |-- PMWindow                   |-- PMWindow
                                 |-- IconWindow
                                       |-- XIconWindow
                                       |-- PMIconWindow
```

`Window` soyut sınıfından X Window ve Presentation Manager için alt sınıflar türetilmiştir. Şimdi pencerenin simge haline getirilmiş (iconified) bir biçimi de isteniyor; görev çubuğuna küçültülmüş ya da köşede küçük simge olarak çalışan pencereler gibi. `Window`'dan yeni bir `IconWindow` türetmek yetmez, çünkü simge haline geçiş X'te ve PM'de farklı kodlanmak zorundadır; `XIconWindow` ve `PMIconWindow` diye iki alt sınıf daha yazmak gerekir. İki pencere yöneticisi daha (GNOME, KDE) eklenirse yalnız iki yeni sınıf eklemekle yetinilmez, simge halleri için iki sınıf daha gerekir. Sonra başka bir pencere çeşidi istenirse (hoca "transparent window" örneğini verdi), onun da her pencere yöneticisi için alt sınıfları yazılır. Bir şey yapmak için birden çok yerde iş yapmak gerekir.

9.3 Çözüm

Temel ilke "neyin değiştiğini bul ve onu sarmala" idi. Burada değişen iki şey vardır: pencerenin soyutlaması (normal pencere, simge haline getirilmiş pencere gibi kavramsal pencereler) ve pencerenin gerçeklemesi (bu kavramsal pencerelerin farklı pencere yöneticilerine uyarlanması). O halde bunlar ayrı kalıtım hiyerarşilerinde sarmalanır. İstemciler soyutlamalarla konuşur; soyutlamalar işi ilgili gerçeklemeye yaptırır. Metot çağrısının soyutlamadan gerçeklemeye iletilmesi, araya bir köprü kurmak olarak ifade edilir; kalıbın adı buradan gelir.

Slayttaki şema (ekrandan, 01:30:00):

```
Window (soyut)                imp      WindowImp (soyut)
  DrawText()            <>--------->     DevDrawText()  (soyut)
```

```
DrawRect()                              DevDrawLine()  (soyut)
  not: imp->DevDrawLine()  (4 kez)
  ^                                       ^
  |-- IconWindow                          |-- XWindowImp
  |     DrawBorder()                      |     DevDrawText()  not: XDrawString()
  |       not: DrawRect(), DrawText()     |     DevDrawLine()  not: XDrawLine()
  |-- TransientWindow                     |-- PMWindowImp
        DrawCloseBox()                          DevDrawLine()
          not: DrawRect()                       DevDrawText()
```

Sol taraf soyutlama, sağ taraf gerçekleme hiyerarşisidir; aralarında duruma göre bire bir ya da bire çok bir sahiplik ilişkisi vardır. Pencere çizmek için pencerenin çerçevesinin (`DrawRect`) ve başlığının (`DrawText`) çizilmesi gerekir. `WindowImp`'te bir cihaza yazı ve çizgi çizdiren soyut metotlar vardır. `Window`'un `DrawRect` metodu işi gerçeklemenin metotları üzerinden yaptırır: dörtgen çizmek için dört çizgi çizilir (koordinatlar sadelik için gösterilmemiştir).

Somut soyutlamalar (`IconWindow`, `TransientWindow`) çizim yaparken üst sınıfın metotlarını kullanır; örneğin `IconWindow`'un `DrawBorder()` metodu `DrawRect()` ve `DrawText()` çağırır. Üst sınıf da işi köprü üyesi üzerinden, hangi pencere sistemiyle ilişkilendirilmişse ona yaptırır. Somut gerçeklemeler soyut metotları gerçekler: `XWindowImp`, `DevDrawText` içinde X'in `XDrawString` hizmetini, `DevDrawLine` içinde `XDrawLine` hizmetini kullanır; `PMWindowImp` de PM'nin ilgili çizgi ve metin hizmetlerini çağırır.

> **Ders sonrası not:** Hoca sözlü anlatımda bu pencere çeşidinden "transparent (şeffaf) window" diye söz etti; slayttaki sınıfın adı `TransientWindow`'dur (geçici pencere, ör. bir iletişim kutusu). Kalıbın mantığı açısından fark yoktur.

Kalıp biçiminde anlatıldığında: bir soyutlama (Abstraction) soyut sınıf olarak ele alınır, gerçekleme kısmı (Implementor) da bir soyut sınıftır ve spesifik gerçeklemelere yönelik metotlar soyut metotlar olarak tanımlanır. Soyutlamadaki işlemler, gerçeklemeye yaptırılacak biçimde kodlanır. Böylece kavramsal özelleştirmeler, gerçeklemeden soyutlanmış olarak `Abstraction`'ın alt sınıflarıyla yapılabilir.

9.4 Uygulanabileceği anlar

Slayttaki maddeler (ekrandan, 01:32:40):

- Bir soyutlama ile gerçeklemeleri arasındaki bağın kalıcı olmasının istenmediği anlarda.
- Hem soyutlamaların hem de gerçeklemelerin ayrı ayrı özelleştirilmesine gerek duyulduğu anlarda.

Hoca ilk maddeyi, bağın çalışma anında değiştirilebilmesinin istendiği durumlar olarak açıkladı. Bu işin belki Strategy gibi başka kalıplarla da yapılabileceğini söyledi; önemli olan kalıpların ayrıntılarını ve çözdükleri sorunları bilip eldeki probleme en uygun olanı seçmektir. Kalıbın asıl derman olduğu dert ikinci maddedir; motivasyon örneği de onun üzerine kurulmuştur.

10. Bridge Uygulama Örneği: İki Çizim Kitaplığıyla Şekiller

10.1 Problem

Hoca, bu örneği bir kaynaktan aldığını ama kaynağı not etmeyi unuttuğunu söyledi. Yine bir çizim programı üzerinde çalışılmaktadır. Şu aşamada dikdörtgenlerle çalışılıyor, ama iki farklı dikdörtgen ailesi vardır. Birinci aile için sol üst ve sağ alt köşelerin koordinatlarını bilmek yeter; dört çizgi çizerek dikdörtgen oluşturulur. Ancak çizgi çekmek için iki ayrı grafik kitaplığı kullanılmaktadır:

- `DP1` kitaplığında `draw_a_line` metodu vardır ve parametre sırası `x1, y1, x2, y2`'dir.
- `DP2` kitaplığında `drawline` metodu vardır ve parametre sırası `x1, x2, y1, y2`'dir.

Burada basitçe Adapter kullanılamaz, çünkü üçgenler, daireler gibi farklı şekiller de olacaktır ve her biri için ayrı adaptör kurmak mantıklı değildir. Problemi "çeşitli şekil soyutlamaları ve bu şekillerin çizimini hayata geçiren farklı çizim kitaplığı aileleri" diye ifade edince Bridge'in sunduğu çözüm zaten akla gelir. Ama önce akla ilk gelen çözüme bakılır.

10.2 İlk çözüm: kalıtımla

Madem iki tip dikdörtgen var, ortak bir üst sınıftan türetilir ve çizim metotları iki farklı biçimde gerçeklenir (ekrandan, 01:40:00):

```
package bridge.solution1;
public abstract class Rectangle {
    private double _x1, _x2, _y1, _y2;
    public void draw( ) {
        drawLine(_x1,_y1,_x2,_y1);
        drawLine(_x2,_y1,_x2,_y2);
        drawLine(_x2,_y2,_x1,_y2);
        drawLine(_x1,_y2,_x1,_y1);
    }
    abstract protected void drawLine ( double x1, double y1,
        double x2, double y2);
}
public class V1Rectangle extends Rectangle {
    protected void drawLine(double x1, double y1,
                double x2, double y2) {
        DP1.draw_a_line(x1,y1,x2,y2);
    }
}
```

`V2Rectangle` de `drawLine` içinde `DP2.drawline` metodunu, DP2'nin istediği parametre sırasıyla çağırır. `draw()` dört köşeyi sırasıyla birleştirir: $(x_1, y_1) \to (x_2, y_1) \to (x_2, y_2) \to (x_1, y_2) \to (x_1, y_1)$.

`drawLine` neden public değil de protected yapıldı? Bu metot nesnenin iç çalışmasıyla doğrudan ilgilidir: dikdörtgenin çizgileri tam olarak nasıl çizeceği, hangi kitaplığın hangi kodunu çağıracağı iç yapıya aittir. Protected ayrıca alt sınıflara aktarımın sağladığı ek avantajı verir. Hoca, metodun soyut olmak zorunda olmadığını, varsayılan bir gerçeklemesi olsa somut da olabileceğini, ama bunun büyük fark yaratmayacağını ekledi.

Bu çözüm dikdörtgen sorununu çözer. Ama mesele şudur: yalnız dikdörtgenlerle yetinilmeyecektir. Çemberler de eklenecek; birinci dikdörtgen ailesiyle birinci çember ailesi, ikinci dikdörtgen ailesiyle ikinci çember ailesi aynı kitaplığı kullanacaktır. Çizim programı da şeklin tam olarak ne olduğunu dert etmeden her şekle "kendini çiz" (ya da dosyaya kaydederken "kendini kaydet") diyebilmelidir. Bunun için dikdörtgen ve çember ortak bir `Shape` üst sınıfında birleştirilir. Slayttaki şema (ekrandan, 01:42:40):

```
Client ----> Shape (soyut)
               +draw()
                 ^
     +-----------+------------+
 Rectangle (soyut)        Circle (soyut)
   +draw()                  +draw()
   #drawLine() (soyut)      #drawCircle() (soyut)
     ^                        ^
     |-- V1Rectangle - - -> DP1     |-- V1Circle - - -> DP1
     |-- V2Rectangle - - -> DP2     |-- V2Circle - - -> DP2

DP1: +draw_a_line(), +draw_a_circle()
DP2: +drawline(),    +drawcircle()
```

10.3 İlk çözümün zayıf yönleri

Hoca tahtada bir üçgen sınıfı ekleyerek sorunu gösterdi: soyut bir `Triangle` sınıfı ve onun `V1Triangle`, `V2Triangle` alt sınıfları gerekir. Bir üçgen eklemek için üç sınıf eklenmiştir.

Soru (hocanın kendine yönelttiği itiraz): Üçgen zaten iki ayrı halde olacağı için iki sınıf yazmak zorunluydu; ek yük yalnız ortak soyut sınıf, yani iki yerine üç sınıf, yüzde elli fazlalık

değil mi? **Cevap:** Haklısınız, ama bir de üçüncü bir kitaplık ailesi gelirse ne olacağını düşünün: `V3Rectangle`, `V3Circle`, `V3Triangle` ve `DP3` kitaplığı. Olaylar iyice dallanıp budaklanır.

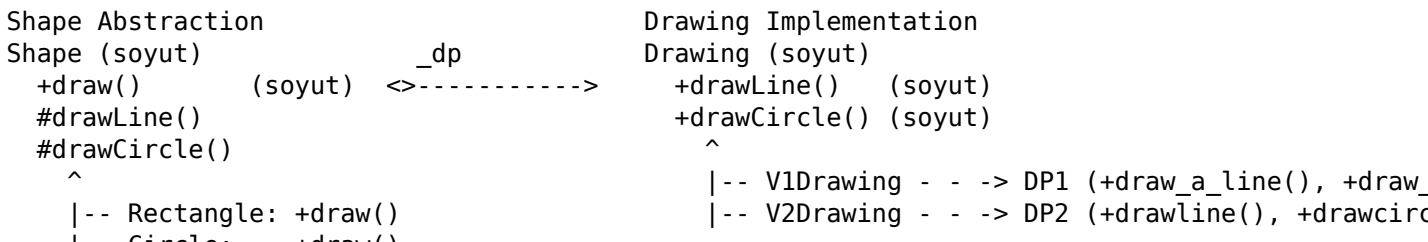
Hoca, belki Abstract Factory'nin de düşünülebileceğini söyledi: aileler çizim kitaplıkları, üyeler üçgen, dörtgen, daire gibi şekiller olur. Ama avantaj ve dezavantajlarının iyi analiz edilmesi gerektiğini, bunu kısa sürede çıkaramayacağını belirtti.

Mevcut durumda iki şekil ve iki kitaplık ailesiyle dört somut şekil sınıfı vardır. Yeni bir çizim kitaplığı eklenirse altı, yeni bir şekil eklenirse yine altı somut sınıf olur. Hoca, her yeni çizim türü ya da şekil türüyle sınıf sayısının katlanarak arttığını söyledi. Sorunun kaynağı, şekil türleri ile çizim yollarının bu ilk çözümde çok sıkı birlikte tasarlanmış olmasıdır. Şekil türlerinin genel soyutlamaları ile onların çizim yollarının (hangi kitaplığın çağrılacağının) gerçeklemeleri birbirinden ayrılırsa sorun çözülür.

> **Ders sonrası not:** Hoca sınıf sayısının "üstel olarak" arttığını söyledi. Doğrusu, somut sınıf sayısının şekil sayısı ile kitaplık sayısının çarpımı kadar olmasıdır (n şekil, m kitaplık için $n \times m$); bu çarpımsal bir artıştır, kelimenin matematiksel anlamında üstel değildir. Bridge ile bu sayı büyük olasılıkla $n + m$'ye iner: örnekte 2 şekil ve 3 kitaplıkla ilk çözümde $2 \times 3 = 6$, Bridge ile $2 + 3 = 5$ somut sınıf olur; fark, şekil ve kitaplık sayısı arttıkça büyür.

10.4 Bridge ile çözüm

Slayttaki şema (ekrandan, 01:50:00):



Soyut `Shape` sınıfının somut alt sınıfları `Rectangle` ve `Circle`'dır; hepsinde `draw()` metodu vardır. `Shape`'te ihtiyaç duyulan temel çizim metotları somut ve protected olarak kodlanır; bu metotların içinden yapılan çizimler köprü üyesi (`_dp`) üzerinden yürütülür. Sağ tarafta gerçeklemenin üst sınıfı olan soyut `Drawing` vardır; çizgi çizmek, daire çizmek gibi soyut metotları bulunur. Somut gerçeklemeler bunları ilgili kitaplıktaki çizim metotlarını çağırarak somutlaştırır.

Böylece `V1Circle`, `V2Circle`, `V1Rectangle`, `V2Rectangle` gibi ayrı ayrı sınıflar kodlamaya hiç gerek kalmaz. Bir şeklin çizilebilmesi için yalnız hangi çizim metotlarının hangi sırayla çağrılacağı, metot adları üzerinden yazılır; bunların hangi aileden hangi kitaplığı kullanacağıyla hiç ilgilenilmez.

Soyutlama tarafının kodu (ekrandan, 01:50:30):

```
package bridge.solution2;
public abstract class Shape {
    public abstract void draw( );
    private Drawing _dp;
    public Shape (Drawing dp) { _dp= dp; }
    protected void drawLine(double x1, double y1,
      double x2, double y2) { _dp.drawLine(x1, y1, x2, y2); }
    protected void drawCircle(double x,double y,
      double r ) {_dp.drawCircle(x, y, r); }
}
public class Circle extends Shape {
    private double _x, _y, _r;
    public Circle (Drawing dp,
              double x,double y,double r) {
        super( dp) ;
        _x= x; _y= y; _r= r ;
```

```
    }
    public void draw () { drawCircle(_x,_y,_r); }
}
```

Soyut `Shape` sınıfının kurucusu, şeklin hangi aileden olacağına göre köprü gerçekleme üyesini atar. `Circle`, merkezin x ve y koordinatlarını ve yarıçapı alır; `Drawing` nesnesini üst sınıfın kurucusuna iletir ve kendi üye alanlarını ilklendirir. `draw()` içinde gerçeklemenin daire çizme metodu çağrılır; gerçekleme hangi aileden verildiyse o biçimde çalışır. `Rectangle` da benzer biçimde iki köşenin koordinatlarını alır, üst sınıf kurucusunu `Drawing` ile çağırır ve `draw()` içinde dört `drawLine` çağrısı yapar.

Gerçekleme tarafının kodu (ekrandan, 01:52:10):

```
package bridge.solution2;
public abstract class Drawing {
    public abstract void drawLine( double x1, double y1,
              double x2, double y2 );
    public abstract void drawCircle (double x,double y,
              double r);
}
public class V1Drawing extends Drawing {
    public void drawCircle(double x, double y, double r) {
        DP1.draw_a_circle(x,y,r);
    }
    public void drawLine(double x1, double y1,
              double x2, double y2) {
        DP1.draw_a_line(x1,y1,x2,y2);
    }
}
```

`V1Drawing`, daire çizileceği zaman DP1'in daire çizme metodunu, çizgi çizileceği zaman DP1'in çizgi çizme metodunu çağırır. `V2Drawing` de aynı işleri DP2'nin metotlarını uygun parametre sırasıyla çağırarak yapar. DP1 ve DP2 sınıflarında bu çizimleri yapan kodlar zaten vardır; onların ayrıntısına girilmez.

11. Çıkarılan Dersler

Hoca, örneğin sonunda "geriye yaslanıp ne deneyim geçirdik, ne dersler çıkardık" diye bir değerlendirme yaptı.

Birinci ders: neyin değiştiğini bul ve onu sarmala. Değişik türden şekiller ve değişik türden çizimler vardır. Bunlar kendi aralarında soyutlanmalıdır: şekil soyutlaması ve çizim soyutlaması.

İkinci ders: kalıtım ilişkisi yerine parça-bütün ilişkisini tercih et. İlk çözümde yalnız kalıtım kullanılmıştı ve kalıtımın yanlış kullanılması, özellikle gereksinimlerdeki değişiklik ve eklemelerde büyük risk oluşturdu: bütün şekil aileleri ve alt türleri her biri kendi alt sınıfıyla modelleniyordu. İkinci çözümde de kalıtım vardır, ama köprü üyesi üzerinden şekil soyutlamasını çizim gerçeklemeleriyle ilişkilendiren bir sahiplik (parça-bütün) ilişkisi de vardır. Her şey kalıtımla çözülmemiştir.

Kalıtım ilişkisi çalışma zamanında değiştirilemez; bir kez kurulduğunda hep öyle devam eder ve değiştirmenin tek yolu kaynak kodu yeniden düzenlemektir. Sahiplik ilişkisi ise çalışma zamanında değiştirilebilir: ilişkideki nesnelerden birinin yerine ötekini atarsınız, olur biter. Bridge örneği bu kuralın getirdiği yararları gösteren bir uygulamadır.

Örnek Sorular

Soru 1: Çerçeve programı bilmediği sınıflardan nasıl nesne oluşturur? (ekrandan, 00:10:00 ve 00:13:00)

Grafik uygulamalarına yönelik mevcut bir çerçeve programı kullanılarak bir müzik besteleme programı yazılacaktır. Çerçevede soyut `Graphic`, soyut `Tool` ve ondan türetilmiş `GraphicTool`

sınıfları vardır. Çerçeve programının nota sınıfları hakkında önceden bilgisi olamaz. `GraphicTool` sınıfı bilmediği sınıflardan çizim nesnelerini nasıl oluşturur?

Derste verilen çözüm: Prototype kalıbı kullanılır. `GraphicTool`, `Graphic` türünden bir `prototype` üyesi tutar. `Manipulate()` içinde `p = prototype->Clone()` ile kopya alınır, kullanıcı fareyi sürükledikçe `p->Draw(new position)` çağrılır, sonunda `p` çizime eklenir. `Graphic`'te soyut `Draw(Position)` ve `Clone()` vardır; `Staff`, `WholeNote`, `HalfNote` gibi somut sınıflar `Clone()` içinde kendi kopyalarını döndürür. Böylece aynı `GraphicTool` sınıfı farklı prototiplerle yapılandırılarak her nota için kullanılabilir.

Soru 2: Müzik örneği Abstract Factory ya da Factory Method ile çözülebilir mi? (derste sözlü, 00:14:44)

Aynı müzik besteleme problemini Abstract Factory ya da Factory Method ile çözmek isteseydik aileler ve üyeler nasıl kurulurdu, sakıncası ne olurdu?

Derste verilen çözüm: Hoca kesin konuşmamakla birlikte, sayısı az olanın aile yapılmasını önerdi: dört nota türü (tam, yarım, ters, düz) aile, yedi nota (do, re, mi, ...) üye olur; örneğin tam nota fabrikasından "do" istenir. Factory Method da benzer kurulabilir. İkisinin sakıncası, her nota ya da nota türü için ayrı fabrika sınıfı gerektirmesidir (dört ya da yedi fabrika); bu, Prototype'a göre daha az kullanışlıdır.

Soru 3: Level editöründe yeni canavar türü oluşturmak (ekrandan, 00:30:00 ve 00:40:00)

Bir oyun için level editörü hazırlanıyor. Haritaya çok çeşitli türlerden canavarları kolayca ekleyebilmek istiyoruz, ama bir canavarın HP, DEX, AGL, LUCK, CHR gibi çok sayıda durum bilgisi var. Bütün değerleri 10 olan genel bir canavardan, canı 100 olan "Bolum sonu canavarı" türü nasıl oluşturulur ve kaydedilir?

Derste verilen çözüm: Prototype kalıbı kullanılır. Soyut `Monster` (Prototype) protected kurucu, soyut `clone()`, public getter'lar ve protected setter'lar içerir; `GenericMonster` (ConcretePrototype) `clone()` içinde aynı statlarla yeni bir nesne döndürür; `MonsterRegistry` canavarları türüne göre bir `HashMap` içinde tutar. İstemci (`MainApp`) önce `new GenericMonster(10, 10, 10, 10, 10)` kaydeder, sonra `reg.findMonster("Generic Monster").clone()` ile kopya alır, `setHP(100)` ve `setType("Bolum sonu canavarı")` yapıp `reg.registerMonster(...)` ile kaydeder. Kod 5.3'tedir.

Soru 4: Adapter neden yetmez, Bridge neden gerekir? (derste sözlü, 01:36:15 ve ekrandan, 01:42:40)

İki dikdörtgen ailesi iki farklı kitaplıkla çiziliyor: `DP1.draw_a_line(x1, y1, x2, y2)` ve `DP2.drawline(x1, x2, y1, y2)`. Metot imzalarını uyumlu hale getirmek için Adapter kullanmak neden mantıklı değildir? Kalıtımla kurulan ilk çözüme bir üçgen ve üçüncü bir kitaplık eklenirse ne olur?

Derste verilen çözüm: Çember, üçgen gibi başka şekiller de olacağı için her şekil için ayrı adaptör kurmak mantıklı değildir. Kalıtımla kurulan ilk çözümde bir üçgen eklemek için soyut `Triangle`, `V1Triangle` ve `V2Triangle` olmak üzere üç sınıf gerekir; üçüncü bir kitaplık gelirse `V3Rectangle`, `V3Circle`, `V3Triangle` ve `DP3` eklenir. Sorun, şekil türleriyle çizim yollarının sıkı birlikte tasarlanmasıdır. Bridge ile `Shape` hiyerarşisi (soyutlama) ve `Drawing` hiyerarşisi (gerçekleme) ayrılır, aralarında `_dp` köprü üyesi bulunur; yeni bir şekil yalnız `Shape` tarafına, yeni bir kitaplık yalnız `Drawing` tarafına bir sınıf ekler.

> **Çözüm (derste verilmedi):** Sınıf sayısı karşılaştırması: n şekil ve m kitaplık için ilk çözümde $n \times m$ somut şekil sınıfı (ve n soyut şekil sınıfı) gerekir; Bridge ile n somut şekil ve m somut çizim sınıfı, yani $n + m$ sınıf yeterlidir. Örneğin 3 şekil (dikdörtgen, çember, üçgen) ve 3 kitaplıkla ilk çözümde $3 \times 3 = 9$, Bridge ile $3 + 3 = 6$ somut sınıf olur.

Soru 5: Bridge kalıbı ne zaman uygulanır? (ekrandan, 01:32:40)

Bridge kalıbının uygulanabileceği anlar nelerdir ve kalıbın asıl çözdüğü sorun hangisidir?

Derste verilen çözüm: Bir soyutlama ile gerçeklemeleri arasındaki bağın kalıcı olmasının istenmediği (çalışma anında değiştirilebilmesi istenen) anlarda ve hem soyutlamaların hem de gerçeklemelerin ayrı ayrı özelleştirilmesine gerek duyulduğu anlarda. Hocaya göre kalıbın asıl derman olduğu dert ikinci maddedir; ilk maddedeki iş belki Strategy gibi başka kalıplarla da yapılabilir.

Dersin Toparlaması

Bu derste yaratımsal kalıpların son ikisi ve yapısal kalıpların ilk ikisi işlendi. Prototype, yeni nesneleri bir prototipin kopyalanmasıyla oluşturur; çerçeve programın önceden bilmediği sınıflardan nesne üretmesini sağlar ve sınıflar çalışma anında belirlendiğinde, paralel factory hiyerarşisi istenmediğinde ya da durum uzayı sınırlı olduğunda Factory Method'a tercih edilir. Kopyalamanın sığ mı derin mi olacağı gereksinime göre seçilir; derin kopyalamada çevrimselliğe dikkat edilir. Level editörü örneğinde protected kurucu ve protected setter'larla nesne oluşturmanın kopyalama ve registry üzerinden yürümesi sağlandı. Singleton, bir sınıfın tek örneğini ve ona ortak erişim noktasını, static üye, private kurucu ve static `getInstance()` ile garanti eder.

Adapter, işlevi uyumlu ama metot adları (ya da parametre sırası) uyumsuz iki sınıfı, araya `Target`'tan türeyen ve çağrıyı ileten bir sınıf koyarak konuşturur; hazır, kaynak kodu olmayan bir daire sınıfının çizim programına uydurulması bunu gösterdi. Bridge ise soyutlama ile gerçeklemeyi ayrı hiyerarşilere ayırıp aralarına bir köprü üyesi koyar; iki çizim kitaplığıyla şekiller örneğinde, kalıtımla kurulan çözümde şekil ve kitaplık sayısıyla katlanan sınıf sayısının bu ayrımla nasıl önlendiği görüldü. Ders, "neyin değiştiğini bul ve sarmala" ile "kalıtım yerine parça-bütün ilişkisini tercih et" kurallarının yeniden vurgulanmasıyla bitti. Sırada büyük olasılıkla yapısal kalıpların devamı (Composite, Decorator) vardır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.